

U

O

W

CSIT314/ISIT950

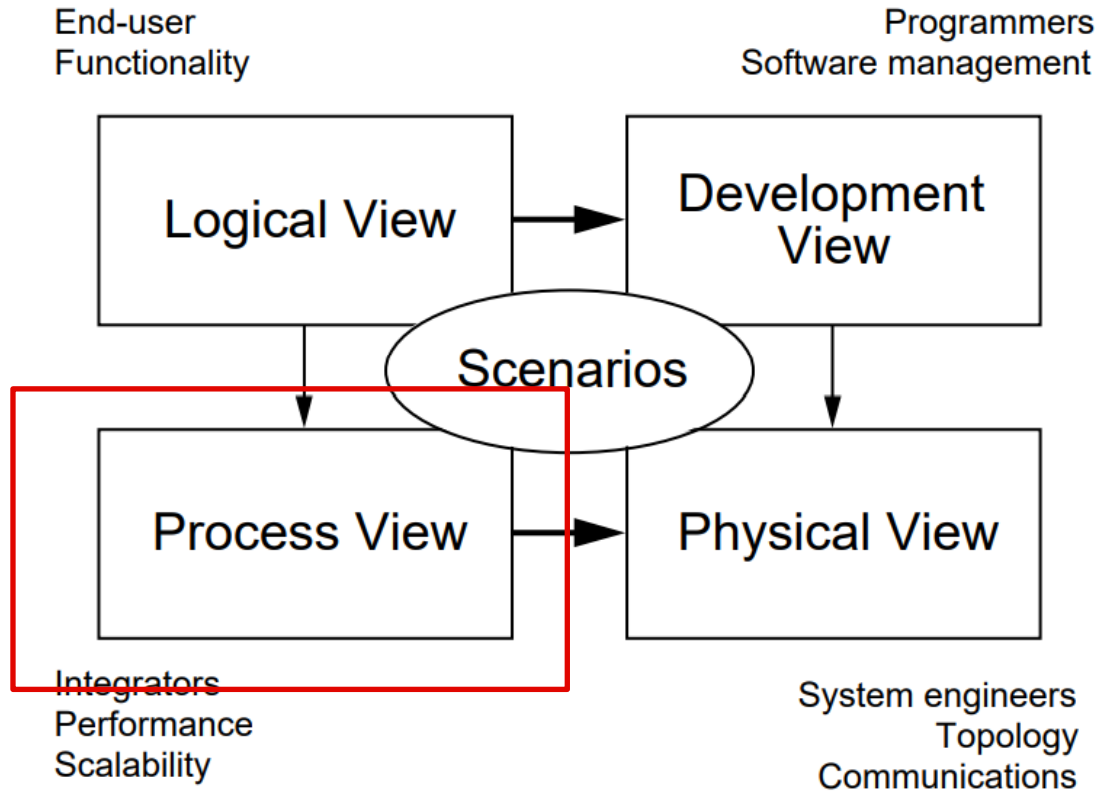
Systems Development Methodologies

Advanced Unified Modelling Language



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

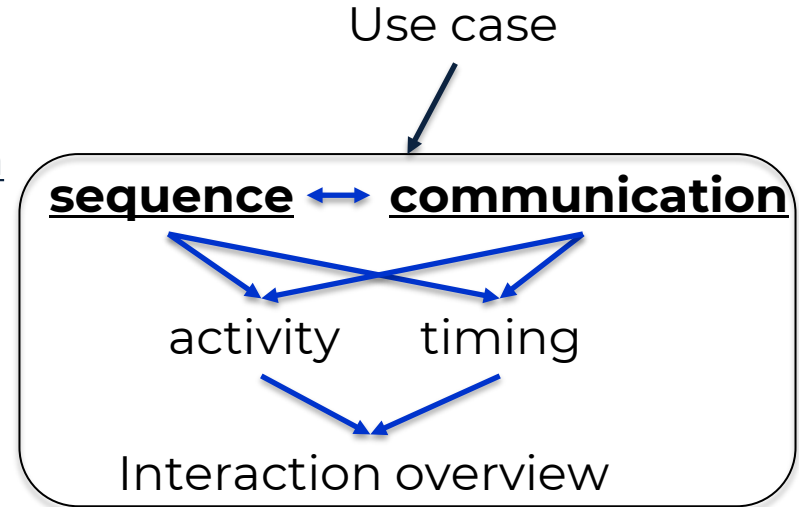
“4+1” View



- Conceptual and Physical
- We discussed logical view so now process view

Process View

- Model the dynamic aspects [runtime behavior] and interaction of the system
 - Either **sequence** or communication diagrams
- Optional:
 - Activity (branching or parallel)
 - Timing
 - Interaction overview (simply other scenarios)



Sequence diagram

- Sequence Diagrams show how **classes/objects** (or groups of objects) collaborate in sequential **order**.
- A Sequence Diagram is often developed after the main classes and relationships have been identified in the Class Diagram, although the two diagrams can also be refined together.
- Generally, one Sequence Diagram describes one Use Case or one possible flow of a Use Case.



Sequence diagram

- Message exchange
 - Communication between objects is achieved by exchanging messages
 - The objects send messages to each other
 - These messages lead to the operations, which means that an object understands or responds precisely those messages for which it has operations



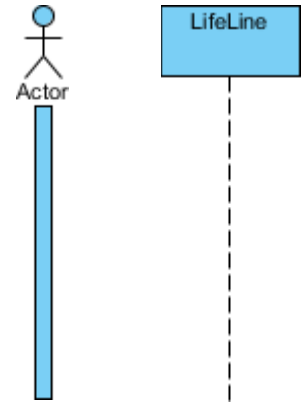
Sequence diagram

- Graphically, a sequence diagram is a figure that:
 - shows **classes/objects** arranged along the **X** axis
 - **messages**, ordered in increasing time, along the **Y** axis.
- The object that initiates the action is placed on the left and other increasingly subordinate objects to the right.



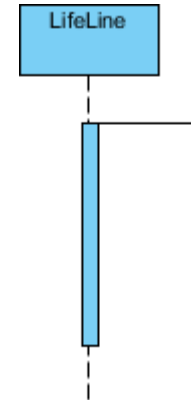
Sequence diagram

- Class/Objects
 - Shows instance name and/or class name (instance_name:class_name)
- A lifeline shows the life of an object
- The X(cross) shows the point at which the object or Actor is deleted

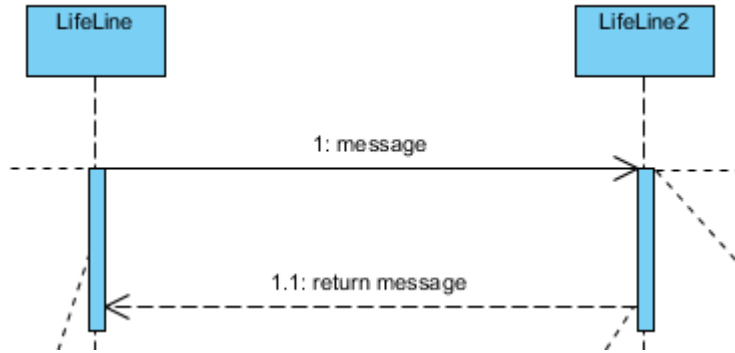


Sequence diagram

- Focus of control [Activations]
 - a long, hollow, narrow rectangle placed over a lifeline
 - shows object has control (sending or receiving messages)
 - The top and the bottom of the of the rectangle are aligned with the initiation and the completion time respectively



Sequence diagram

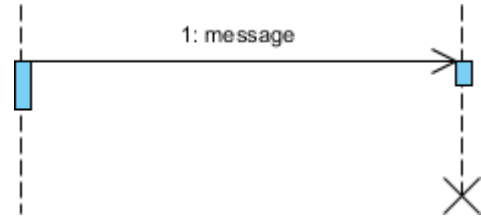
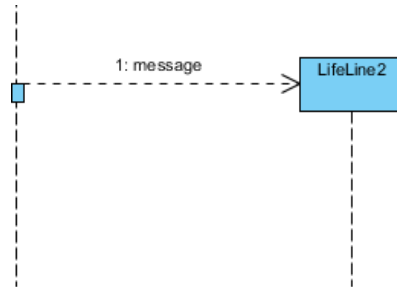
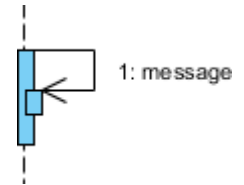
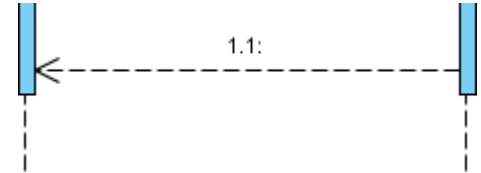
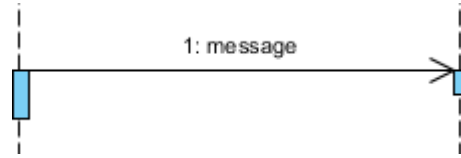


- Message
 - Shown by a line with a filled in arrow head
 - Message name as the class method
 - Private methods as messages



Sequence diagram

- Call
- Return
- Self
- Recursive
 - One special self
- Create
 - Controller → Order: createOrder()
- Destroy
 - Controller → Session: logout()



So far...

- Use case: system functionality or user requirement (what)
- Class/object: implement use case, structure (who)
- Sequence : events change object status, or behaviour (how)
- Code: implementation



ATM Example

ATM

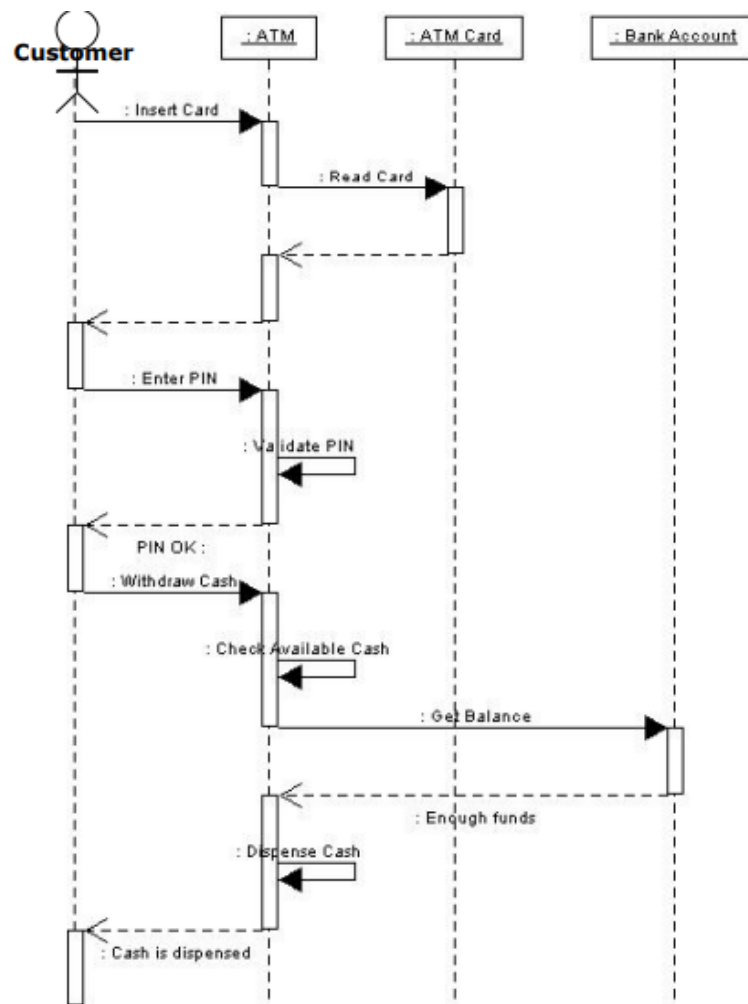
- +InsertCard()
- +EnterPIN()
- +ValidatePIN()
- +CheckAvailableCash()
- +WithdrawCash()
- +DispenseCash()

ATMCard

- +ReadCard()

BankAccount

- +GetBalance()



Problems

- **Too many responsibilities** are placed in a single class (ATM)
- User interaction, business logic, data checking, and device control are **mixed** together
- The class becomes tightly coupled and **harder to modify**
- The design is difficult to **test, maintain, and extend**
- **So how can we organise responsibilities more clearly?**



B-C-E design

- Consider applying the B-C-E model
 - Boundary layer
 - Responsible for representing info to user and to **receive user interactions**.
 - User interface (like front-page, mobile app)
 - Control layer
 - Contains the rules about how to combine info and how to deal with **interaction**
 - A third-party coordinator(sequence diagram)
 - Entity layer
 - Holds the **data** and is responsible for its persistence.
 - **Class, attributes, methods**



B-C-E design

- Boundary layer
 - The interaction between the user and the system
 - A boundary class **does not provide** the actual behavior of the use case
 - It typically represents the **GUI** - e.g. dialog, menu, screen.
 - *Actors interact only with boundary classes, to perform the use case*



B-C-E design

- Control layer
 - The main **use case controller** class to coordinates all the messaging between the entity classes
 - They are the “glue” between boundary and entity classes – i.e. they **connect the user with the data and logic**
 - They contain the application-specific **business rules**
 - A student should enroll in 4 subjects per semester; a customer cannot withdraw more than \$800 per day.
 - One control class **per use case** (some use cases may not require one)
 - Created when the use case starts and typically destroyed when it ends



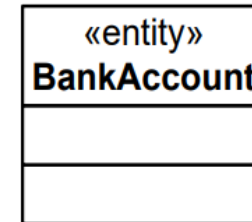
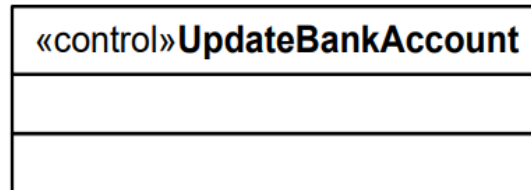
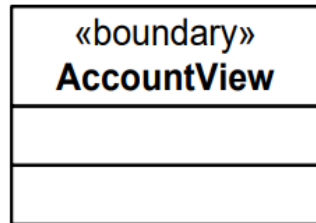
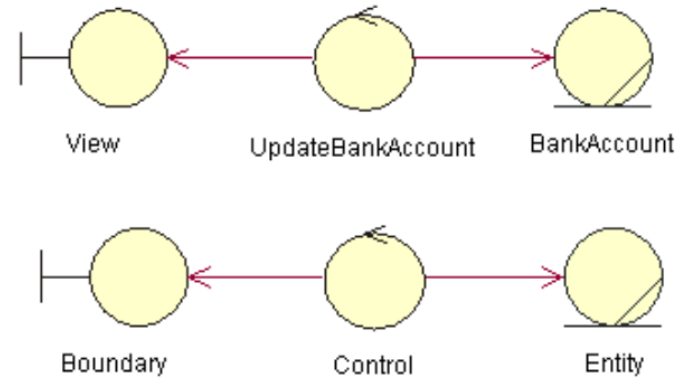
B-C-E design

- Entity layer
 - Represents core business domain concepts identified from the **class** diagram
 - Stores and manages system **data**
 - Many entity classes are persistent and may map to **database** tables
 - Some entity classes may be transient, meaning they **exist only temporarily** during system operation

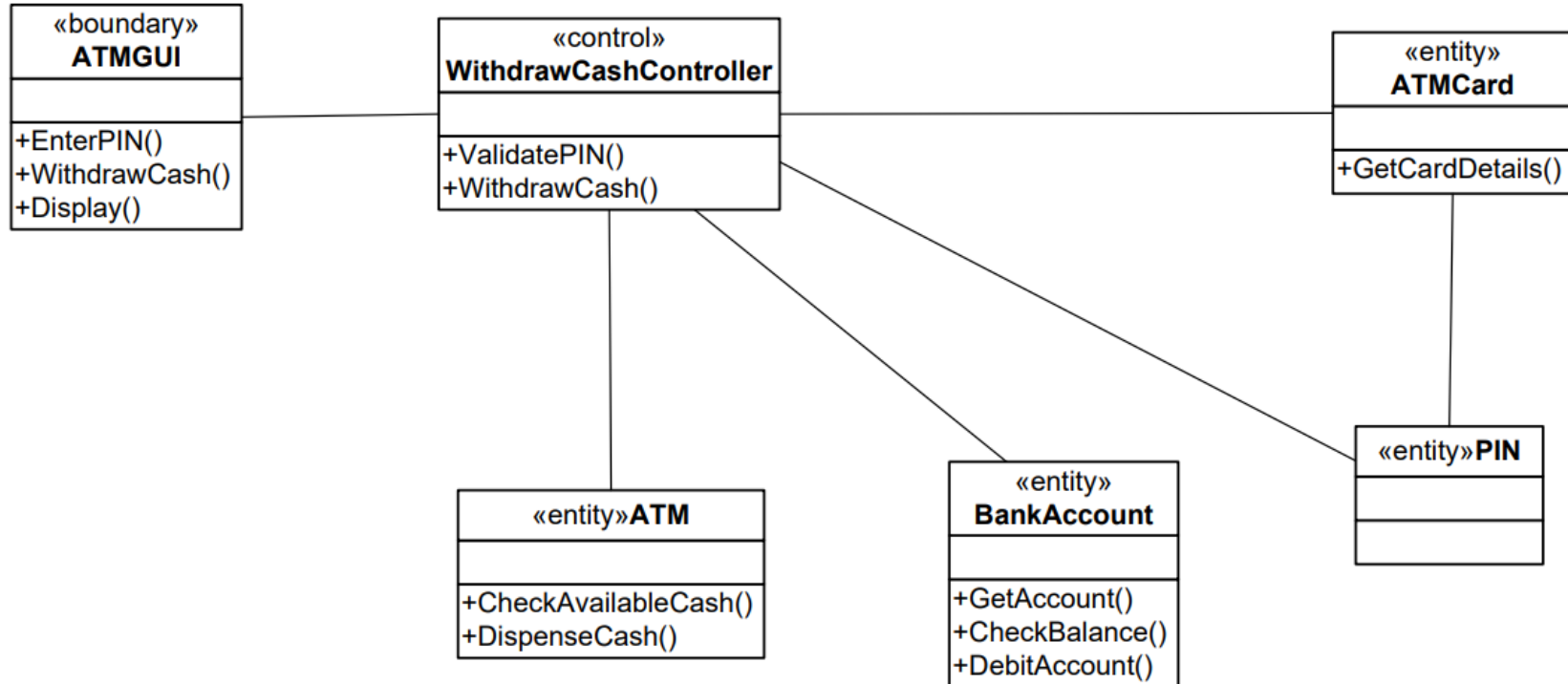


B-C-E design

- 2 representations:
 - Circle for class, line for association between class
 - Add prefix



ATM B-C-E design



B-C-E design

- Applying the B-C-E framework to ATM

Without BCE (Traditional Design)

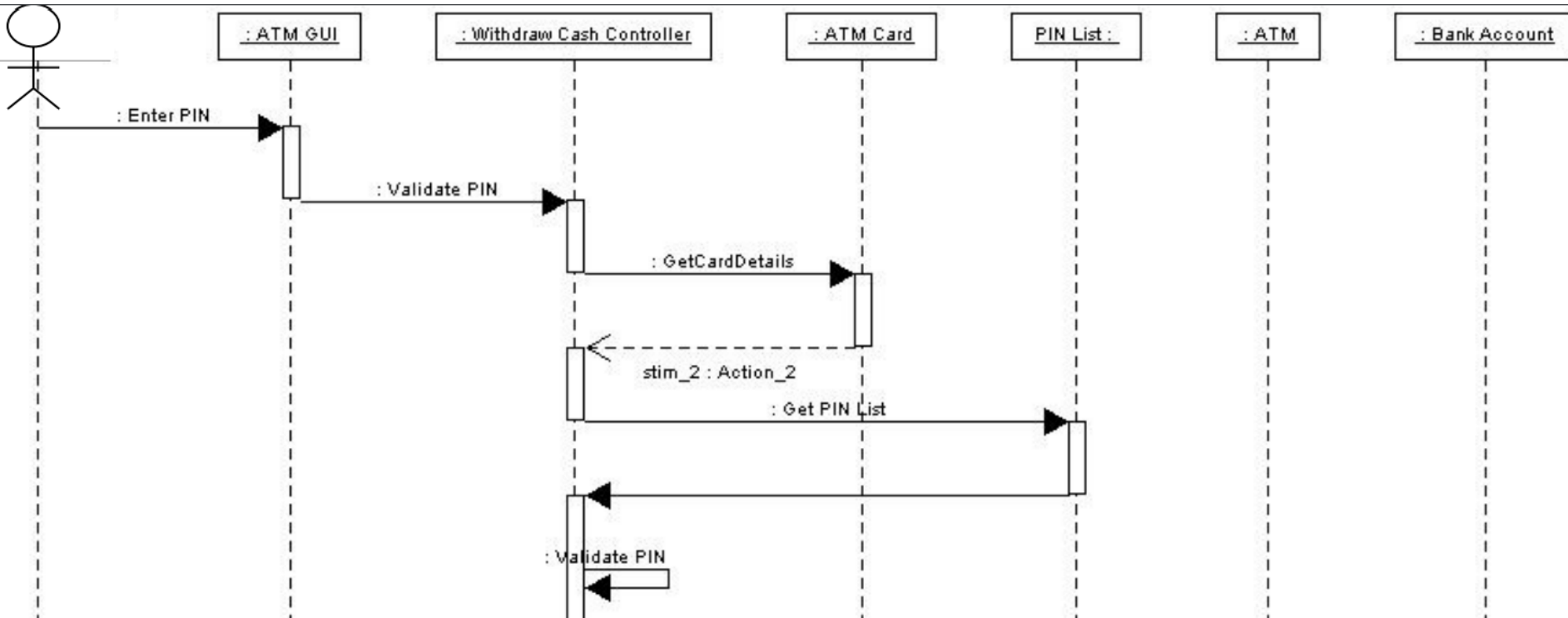
- Methods are distributed across classes without clear responsibility
- Business logic is mixed with interaction and system operations
- No central component to manage the use case flow
- Responsibilities are unclear and tightly coupled
- Difficult to understand, maintain, and extend

With BCE

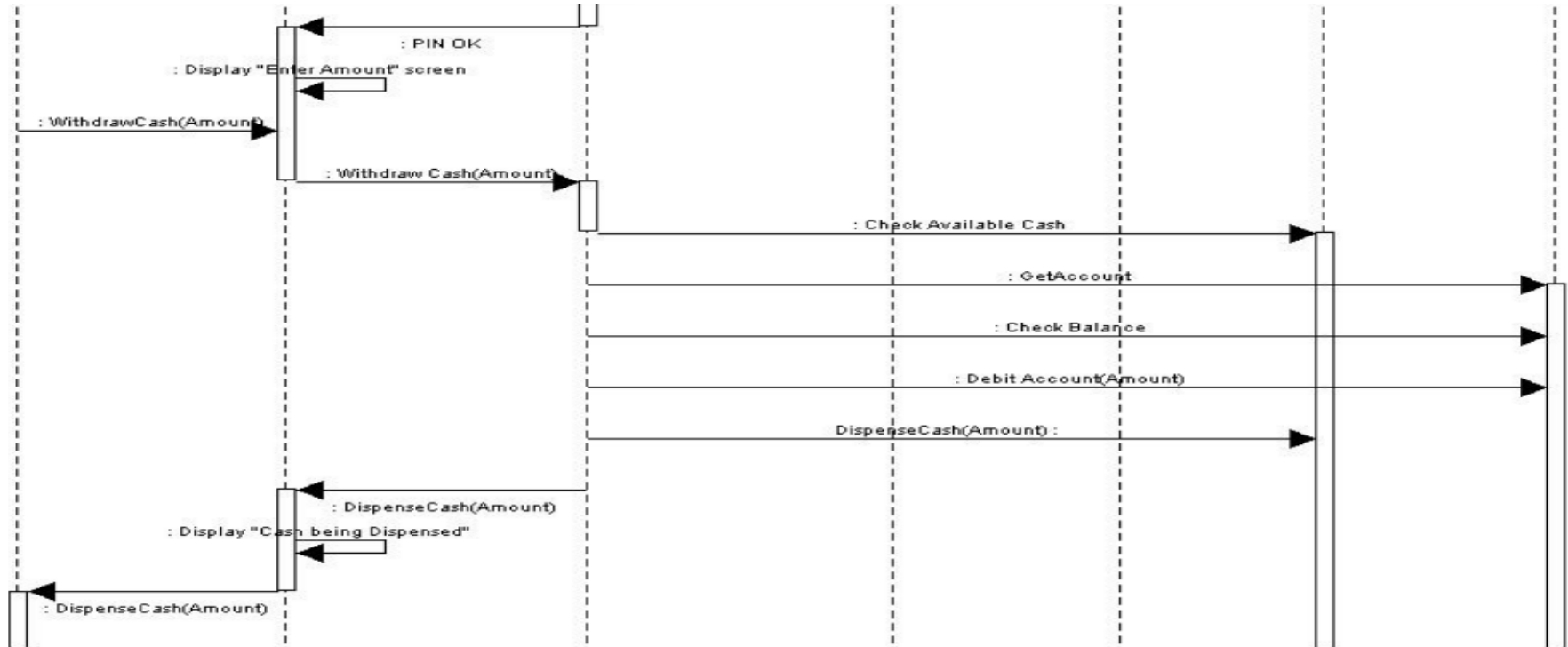
- Responsibilities are clearly separated into three roles:
 - **Boundary**: handles user interaction
 - **Control**: manages the use case workflow
 - **Entity**: represents data and business logic
- A central control object coordinates the process
- Clear interaction flow: **Boundary** → **Control** → **Entity**
- Improved modularity, readability, and maintainability



ATM B-C-E design



ATM B-C-E design

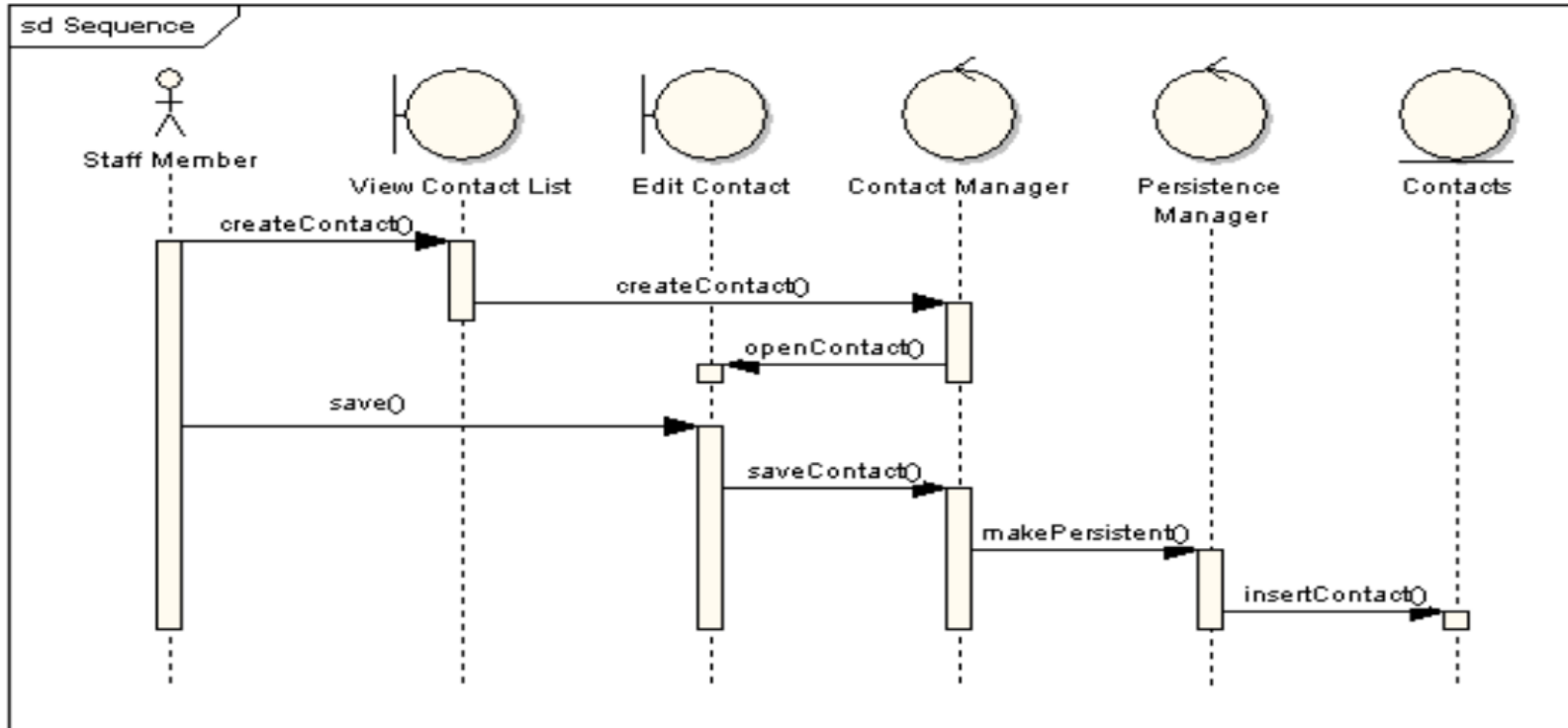


Process View: Communications diagrams

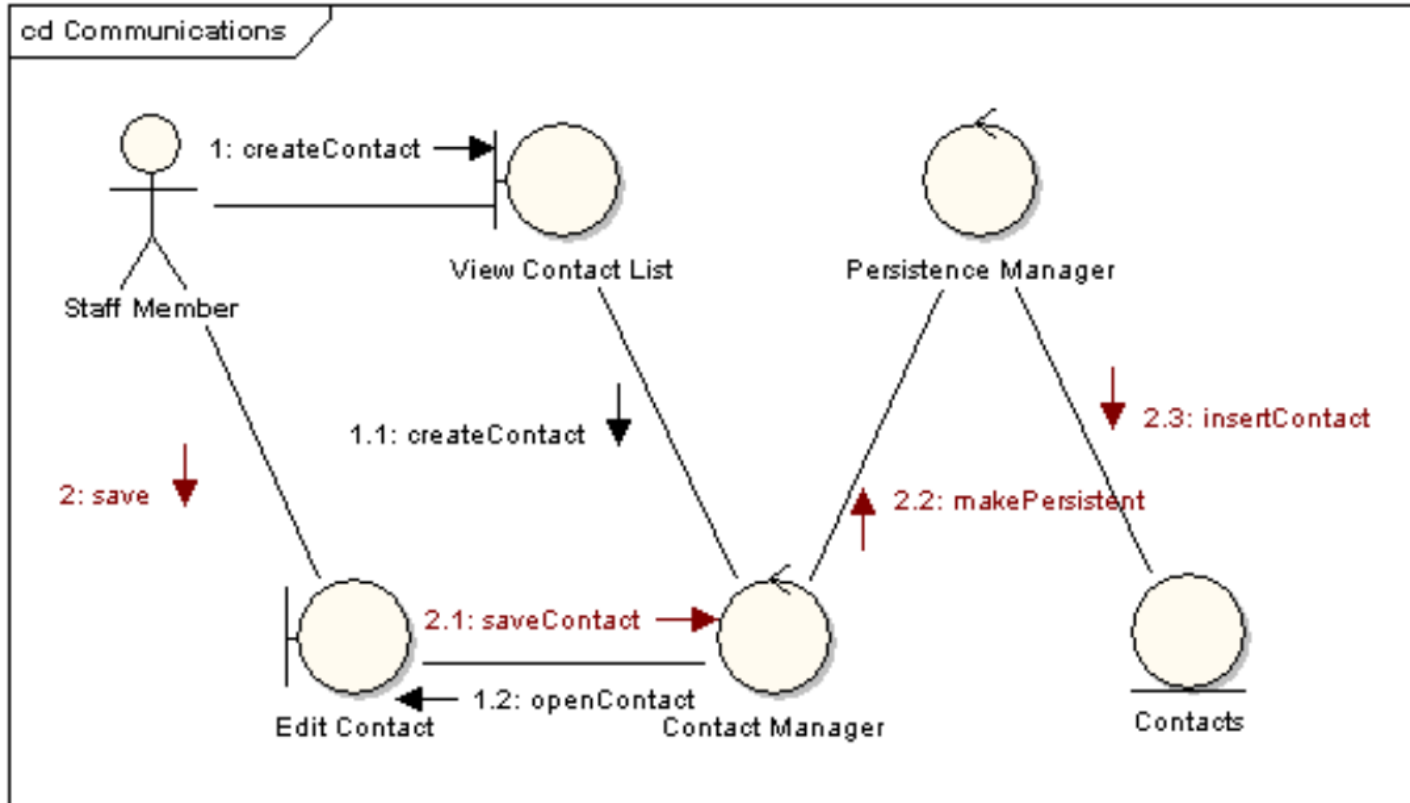
- Convey the same information as a sequence diagram – a difference in style.
 - Sequence diagrams focus on message **order** occurring over a timeline
 - Communication diagrams focus more on links between participating **objects**
 - sequence is less clear (order info can be unclear) but relationships are more clear



Sequence vs. Communications



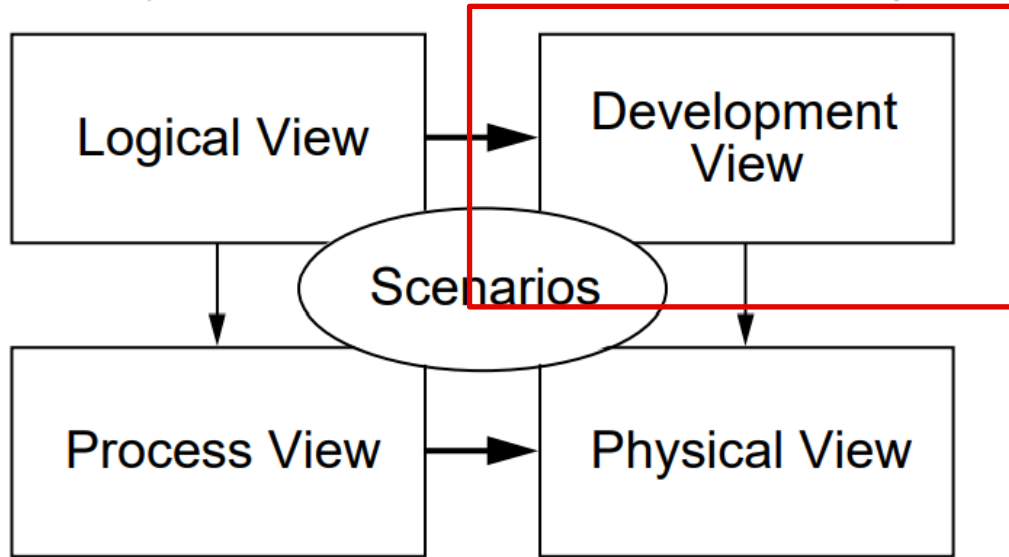
Sequence vs. Communications



“4+1” View

End-user
Functionality

Programmers
Software management



Integrators
Performance
Scalability

System engineers
Topology
Communications

- Conceptual and Physical
- We discussed conceptual view so now physical view

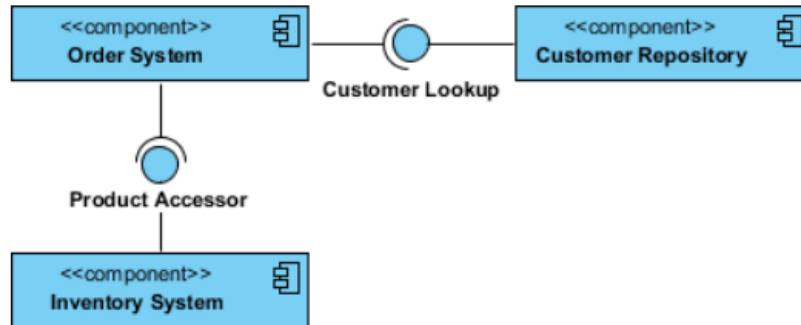
Development View

- Component diagrams illustrate the pieces of software, embedded controllers, etc., that will make up a system.
 - show the relationship between different parts
 - organise the system (show subsystems)
- The term "component" refers to a module of classes that represent independent systems or subsystems with the ability to interface with the rest of the system.
- A component is usually implemented by **one or more classes (or objects)** at runtime.



Component diagrams

- Component: rectangles
 - Class or (sub)system needs to communicate with others to perform something (provide services)
 - Assembly Connector



Package Diagram vs Component Diagram

Aspect	Package Diagram	Component Diagram
Focus	Logical organisation	Physical/implementa- tion structure
Level	Design-time	Implementation/ru- ntime
Elements	Packages (group of classes)	Components (modules/services)
Purpose	Structure code	Show system architecture
Example	UI, Service, Data packages	Web Server, Database, Payment Service

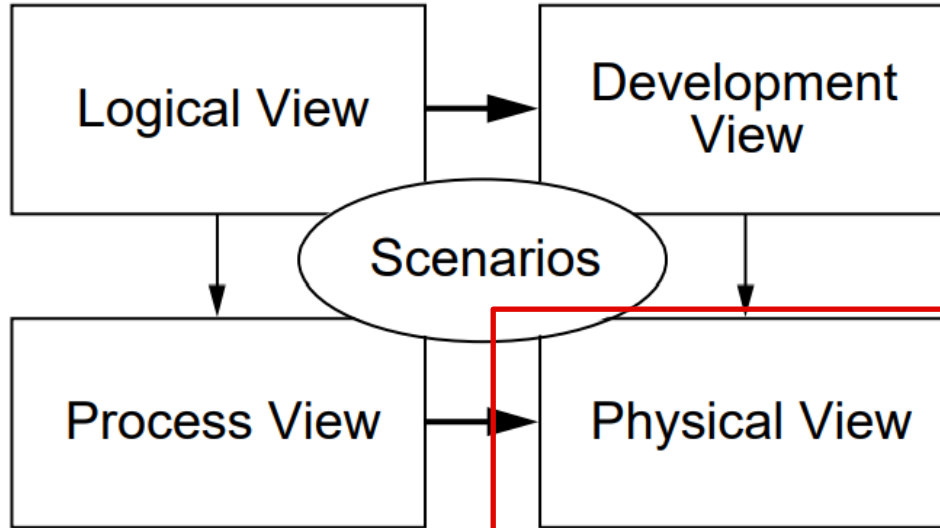
- Package = how code is organised
- Component = how system is built and deployed



“4+1” View

End-user
Functionality

Programmers
Software management



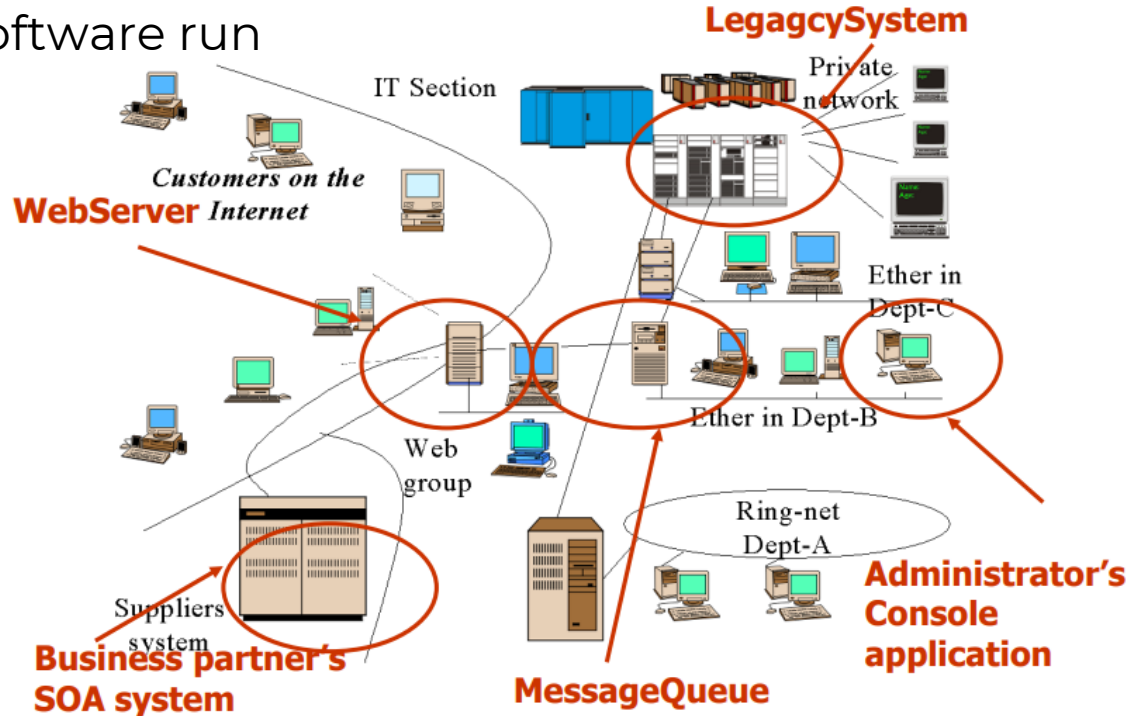
- Last view

Integrators
Performance
Scalability

System engineers
Topology
Communications

Physical view

Where do software run



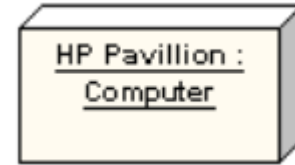
Deployment Diagrams

- Deployment Diagrams
 - show the physical hardware nodes and their connections
 - show how software artifacts or components are deployed on these nodes
- A Node is a **hardware or artifact** element, shown as a three-dimensional box shape



Deployment diagram

- Node **Instance**: Name and base node type
- Node **types**
 - standard stereotypes are provided for nodes, «cdrom», «cdrom», «computer», «disk array», «pc», «pc client», «pc server», «secure», «server», «storage», «unix server», «user pc».



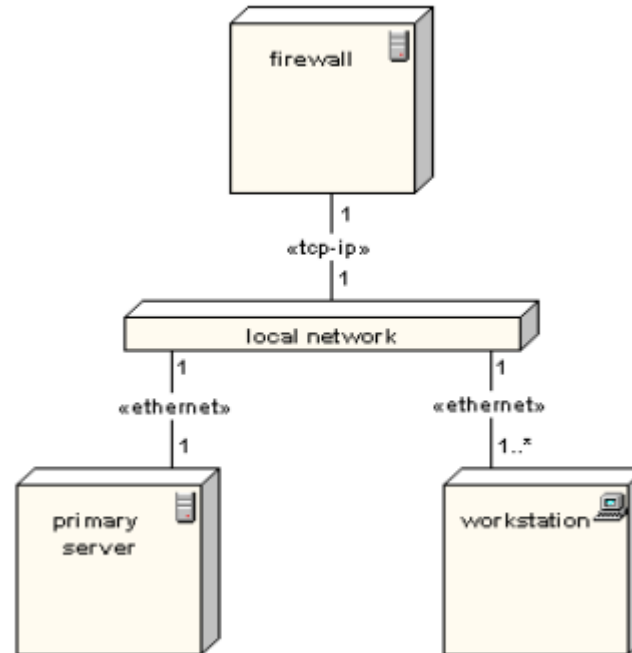
Deployment diagram

- Artifact (soft parts)
 - Any product of the software development: source files, executables, design documents, test reports, prototypes, user manuals, etc.
 - An artifact is denoted by a rectangle showing the artifact name, the «artifact» keyword and a document icon.



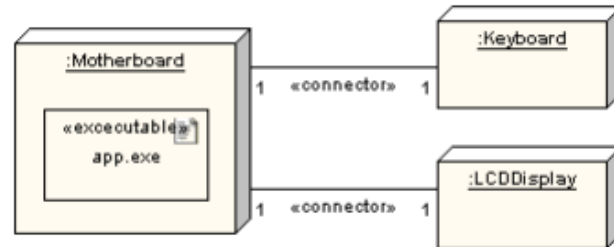
Deployment diagram

- Association
 - An association represents a communication path between nodes



Deployment diagram

- Node as Container
 - A node can contain other elements, such as components or artifacts.



Deployment diagram

